Momentum Dashboard (Indian Stocks) — Developer & Solution Design Guide

Version: 2025-10-04
Scope: One stop, end-to-end guide for functional + technical details, architecture, APIs, data model, scoring, flows, runbook, and developer tooling.

> This guide is based on the source tree you shared (`momentum-dashboard.zip`) plus your blueprint and functional notes. Contract and architecture claims are cross-referenced to your blueprint and requirement docs where relevant.

---

1) System Overview

The Momentum Dashboard is a **local-first** swing-trading research suite for Indian equities. Architecture is split into a **FastAPI** backend (Python) and a **React + TypeScript** frontend. Storage is **hybrid**: columnar **Parquet** for market/universe/indicator/score snapshots and **SQLite** (WAL) for user/state data (alerts, positions, watchlist, jobs, settings). YAML-first configuration with environment overlays, OpenAPI-first contracts with generated clients, and an in-process **APScheduler** drives recurring scans. □filecite□turn1file0□L1-L13□ □filecite□turn1file0□L31-L46□

High-level Data Flow

React SPA → generated OpenAPI client → FastAPI services → Repos/UoW → Parquet (universe/prices/indicators/scores) + SQLite (alerts/watchlist/history/jobs/settings) → Workers/Scheduler → Notifications. □filecite□turn1file0□L11-L23□

Why Parquet + SQLite

Parquet scales for **analytical** columnar reads across thousands of symbols and many metrics, while SQLite stays small/transactional for **user CRUD** and app state. □filecite□turn1file6□L61-L72□

---

2) Goals & Functional Requirements (condensed)

- **Find and rank momentum candidates** with a 0–100 score; show badges, meters (risk/euphoria), and **actionable next steps** in a Right Drawer. □filecite□turn1file1□L11-L21□
- **Screener** with filters/sort/pagination; **manual scans**; **runs** history. □filecite□turn1file4□L52-L60□ □filecite□turn1file7□L97-L106□
- **Positions/Locks**, **Alerts**, **Watchlist**, **History/Replay**, **Settings**, **Learning** pages. □filecite□turn1file5□L64-L73□ □filecite□turn1file10□L15-L23□ □filecite□turn1file3□L66-L74□
- **Digest/Notifications** with de-dupe and channels. □filecite□turn1file8□L1-L8□

A fuller UI content map (KPIs, heatmap, top gainers, drawer examples) is in your functional draft. □filecite□turn1file3□L33-L47□ □filecite□turn1file3□L51-L63□

---

3) Package & Repo Structure (source tree)

Backend (`backend/app/`)

-
- ". ".join(backend_highlights)

Frontend (`frontend/`)

-
- ". ".join(frontend_highlights)

Contracts (`/contracts`) — OpenAPI 3.1, examples for all payloads; FE types/client generated from here (CI guard). □filecite□turn1file4□L21-L33□ □filecite□turn1file4□L70-L76□

---

4) APIs (contract-first)

Core:

- `GET /health`, `GET /health/live`, `GET /health/ready`
- `POST /scan` (idempotent via `Idempotency-Key`)
- `GET /screener` (filters/sort/pagination; supports `run_id` and `as_of`)
- `GET /runs`, `GET /runs/{run_id}`
- `GET /instruments/{symbol}/detail` (+ optional `/prices`, `/indicators` slices)
- `GET/PUT /settings`
- `GET/POST/PUT/DELETE /alerts`, `/watchlist`, `/positions`
- `POST /snapshots/pin`, `GET /snapshots/pins`, `DELETE /snapshots/pin/{id}`
- `GET /universe`, `/universe/sectors`
- `GET /history`, `GET /learning`
  □filecite□turn1file4□L52-L60□ □filecite□turn1file5□L46-L60□ □filecite□turn1file10□L1-L13□ □filecite□turn1file10□L15-L31□ □filecite□turn1file10□L53-L67□ □filecite□turn1file12□L1-L6□

HTTP semantics & consistency: ETag/If-None-Match for reads; standardized `run_id` (UTC `YYYYMMDDHHmmss`), `as_of` (ISO Z), and `symbol_canon`. □filecite□turn1file4□L76-L84□

FE usage: Components call these via generated client/hooks; manual scan triggers `/scan` then refetch. □filecite□turn1file10□L69-L79□

---

5) Screening Flow (end-to-end)

1. Universe slice (preset or custom) → 2) Indicators & stats → 3) Score → 4) Write snapshot (Parquet) with `run_id` → 5) Update SQLite summaries → 6) Post-scan jobs (alerts) → 7) Frontend renders `/screener` for the latest or selected `run_id`. □filecite□turn1file9□L39-L48□ □filecite□turn1file11□L39-L49□

Scheduler: Interval trigger (e.g., every 15m), coalesce, single instance; each run emits a `run_id` used as snapshot partition. □filecite□turn1file0□L35-L43□

---

6) Scoring & Rules

You support **Basic** and **Full** scoring modes.

6.1 Basic (0–12 → 0–100%)

Piecewise mapping for RSI, ADX+slope, breakout quality, and volume/OBV; final is scaled to a percentage. **Buy** if ≥ ~60%. □filecite□turn1file8□L55-L63□ □filecite□turn1file8□L69-L108□

6.2 Full (0–100)

Weighted pillars: **Momentum** (RSI+ADX) 35, **Breakout Quality** 30, **Accumulation/Volume** 25, **Market/Sector Context** 10; smooth mappings per pillar. □filecite□turn1file8□L117-L127□ □filecite□turn1file8□L131-L147□

6.3 Drawer Metrics & Next Action

Right Drawer exposes price/1D%, returns (1M/3M/6M/12-1M), indicators (RSI, ADX, EMAs, ATR%, 52W proximity, RelVol), score/badges, meters (risk/euphoria), **entry block** (entry/stop/breakeven/lock), and **next_action** states with reasons/hints. □filecite□turn1file1□L5-L21□

---

7) Data Model & Storage

7.1 Parquet datasets (append-only)

Folders: `universe/`, `prices/`, `indicators/`, `scores/`, `meta/`. Partitioning by `run_id` (and `dt` where relevant). Snapshot layout: `{parquet_dir}/{table}/run_id={{YYYYMMDDHHmmss}}/dt=YYYY-MM-DD/*.parquet` with `_SUCCESS`, `rowcount.txt`, and schema metadata. Atomic write: temp dir → write → rowcount → `_SUCCESS` → atomic promote + file lock. □filecite□turn1file4□L39-L43□ □filecite□turn1file11□L63-L76□

Compression: zstd with dictionary & statistics enabled (configurable). □filecite□turn1file11□L69-L76□

Lineage: store `schema_version`, `source_version`, and `run_id` in file metadata; surfaced by APIs. □filecite□turn1file0□L53-L55□

7.2 SQLite tables (user/state)

- **alerts** — User-configured alert rules and channels
- **watchlist** — Symbols pinned for quick access
- **history** — Historical outcomes / summaries linked to run_id
- **jobs** — Background job runs; run_id, timings, status
- **settings** — YAML-backed app settings snapshot
- **positions** — Right Drawer 'Lock' entries: entry price, stops, P&L trail
- **snapshot_pins** — Per-symbol pinned run_id for stable comparisons

Schema keys: `history.run_id` ↔ `scores.run_id` for traceability; symbol canonicalization in Parquet `universe/`. □filecite□turn1file6□L47-L56□

---

8) Repository Layer (how data is written/read)

- **Parquet** — `app/repos/parquet/datasets.py` provides `begin_atomic_write`, `latest_snapshot`, `scan(...)`, and schema version helpers. `ScoresRepo` reads the latest/selected `run_id` and enriches from `universe/`.
- **SQL** — repos under `app/repos/sql/*` wrap CRUD with session management; `jobs_repo` issues UTC `run_id` (`YYYYMMDDhhmmss`). □filecite□turn1file11□L107-L135□

Single-writer guard: file lock during parquet promotion; DB uniqueness prevents duplicate `run_id` rows. □filecite□turn1file9□L1-L3□

---

9) Market Data (Yahoo) & Sparklines

`MarketDataRepo` uses **yfinance** with a small TTL cache to serve recent closes and sparkline-friendly arrays (prices + aligned ISO dates). It also includes an in-process EMA helper. (See: `app/repos/market_data_repo.py`).

---

10) Schedulers & Jobs

- **APScheduler BackgroundScheduler** starts with the app; jobs are configurable via YAML.
- **Recurring scan**: creates a new `run_id`, writes Parquet snapshot, updates SQLite summaries, logs rowcount/timings, and invokes **post-scan jobs** (e.g., momentum cross-up alerts, digests). □filecite□turn1file0□L31-L45□

Retention/GC is **configurable**; recommended **tiered** policy: keep all intraday runs for 7 days, daily for 90 days, weekly for 6 months, monthly for 2 years; pins protect specific `run_id`s. □filecite□turn1file7□L109-L137□

---

11) Frontend Contract & Structure

The FE is **contract-first**. Generate `src/lib/api/types.ts` (+ client) from OpenAPI, then wire Screens/Table/Drawer strictly to generated types—no handcrafted DTOs. Components & routes are scaffolded to mirror the dashboard UX. □filecite□turn1file2□L81-L99□ □filecite□turn1file2□L101-L111□

---

12) Running the Project (local)

1. **Python env**: `python -m venv .venv && source .venv/bin/activate` (Windows: `Scripts\activate`); `pip install -r backend/requirements.txt`
2. **Config**: copy `configs/development.yaml` as base; override via env (`APP_ENV`) if needed.
3. **DB**: initialize SQLite (WAL on, FKs on; Alembic migrations if present).
4. **Start API**: `uvicorn app.main:app --reload --port 8000` from `backend/` root.
5. **FE**: `pnpm i && pnpm dev` (or yarn/npm) from `frontend/` root; set `VITE_API_BASE` to `http://localhost:8000/api/v1`.
6. **Scheduler**: enabled via YAML; verify logs show interval job registered.
7. **Manual scan**: `POST /api/v1/scan` → then open `/api/v1/screener?run_id=...`. □filecite□turn1file11□L91-L96□

CLI helpers: `backend/util/fetch_screener_pages.py` can export large screener pages (NDJSON/CSV) from the running API.

13) Auto-generated Schemas & Clients

Pydantic models are **generated** (contract-first) for Screener and Drawer payloads; the FE **types/client** are generated from `/contracts/openapi.yaml`. CI can fail on OpenAPI drift until codegen outputs are updated/committed. □filecite□turn1file4□L21-L33□ □filecite□turn1file4□L70-L76□

14) Why keep ~1 year of Parquet snapshots?

- **Backtesting & learning:** replay breakouts/history by `run_id` and study outcomes week-over-week.
- **Model evolution:** re-score or compute new indicators from historical snapshots without refetching.
- **Diagnostics:** trace data lineage (`schema_version`, `run_id`) for any displayed row.
  A tiered policy is recommended to cap disk use while preserving learnings. □filecite□turn1file7□L109-L137□ □filecite□turn1file11□L69-L76□

15) Non-functional & Cross-cutting

- **Validation:** Pydantic at API edge; domain logic stays functional.
- **Idempotency:** `Idempotency-Key` on POSTs.
- **Logging:** request/correlation IDs; latency; rows processed; `run_id`.
- **Time/TZ:** store UTC; UI localizes.
- **Testing:** seeds & determinism; MSW/Playwright for FE. □filecite□turn1file0□L15-L25□ □filecite□turn1file0□L21-L24□ □filecite□turn1file4□L9-L15□

16) Database Entities (SQLite)

- **jobs**: `id`, `name`, `key`, `run_id`, `status`, `started_at`, `finished_at`, `error`
- **positions**: `symbol`, `entry_price`, `entry_dt`, `stop`, `breakeven_on`, `trade_on`, notes
- **alerts**: `symbol`, `rule_type`, `rule_value`, `channels[]`, `enabled`
- **snapshot_pins**: `symbol` → `run_id`
- **watchlist**, **settings**, **history** (summaries)
  (See `app/repos/models.py` for exact fields; align with your migrations.)

17) Troubleshooting & Guardrails (selected)

- Use **temp DB** in tests; dispose engine on shutdown; NullPool for Windows.
- `session.flush()` after writes before reads in the same request path.
- Return **Problem+JSON** consistently for errors; dynamic status for alerts.
- Parquet read helpers must only read after `_SUCCESS` and verify `rowcount`. □filecite□turn1file11□L1-L8□ □filecite□turn1file11□L29-L38□ □filecite□turn1file11□L87-L96□

18) Roadmap Alignment (phases excerpt)

Phase 11–14 bring indicators + full score, Drawer, Alerts/Digest, and Scheduler/Retention; subsequent phases polish Watchlist/History/Positions UX and Settings/Learning. □filecite□turn1file9□L39-L51□

---

19) Appendix — Frontend file map (from blueprint)

See the proposed file tree and generation steps to keep FE strictly in sync with OpenAPI.
□filecite□turn1file2□L89-L101□ □filecite□turn1file2□L103-L119□